

PRÁCTICA 1.3: SOLUCIONES ACTIVIDADES EN GIT/GITHUB

Implantación de Aplicaciones Web (IAW)

CFGS ASIR

Mara Vañó Alonso

Curso 2024-2025

Índice

Índice	2
GIT / GITHUB	3
1. Ejercicios de manejo del historial de cambios.....	3
Ejercicio 1	3
Ejercicio 2	3
Ejercicio 3	4
Ejercicio 4	4
Ejercicio 5	5
Ejercicio 6	5
2. Ejercicios de manejo del historial de cambios.....	6
Ejercicio 1	6
Ejercicio 2	6
Ejercicio 3	7
Ejercicio 4	7
3. Ejercicios de deshacer cambios	8
Ejercicio 1	8
Ejercicio 2	8
Ejercicio 3	9
Ejercicio 4	10
4. Ejercicios de gestión de ramas	11
Ejercicio 1	11
Ejercicio 2	11
Ejercicio 3	12
Ejercicio 4	12
Ejercicio 5	13
5. Ejercicios de repositorios remotos	14
Ejercicio 1	14
Ejercicio 2	15
Ejercicio 3	15
Ejercicio 4	16

GIT / GITHUB

1. Ejercicios de manejo del historial de cambios

Ejercicio 1

1. Abrir la terminal.

Usar la terminal de tu sistema operativo (Linux, macOS o Windows con Git Bash).

2. Configurar el nombre del usuario:

```
git config --global user.name "Tu Nombre"
```

3. Configurar el correo electrónico:

```
git config --global user.email "tuemail@example.com"
```

4. Activar el coloreado de la salida:

```
git config --global color.ui auto
```

5. Mostrar la configuración final:

```
git config --list
```

Ejercicio 2

1. Crear un nuevo directorio para el repositorio:

```
mkdir libro  
cd libro
```

2. Inicializar el repositorio:

```
git init
```

3. Mostrar su contenido (debería estar vacío):

```
ls -la
```

Ejercicio 3

1. Comprobar el estado del repositorio:

```
git status
```

2. Crear un fichero indice.txt con el contenido especificado:

```
echo -e "Capítulo 1: Introducción a Git  
  
Capítulo 2: Flujo de trabajo básico Capítulo 3: Repositorios remotos" >  
indice.txt
```

3. ****Comprobar de nuevo el estado del repositorio:**** ``bash

```
git status
```

4. Añadir el fichero a la zona de intercambio temporal:

```
git add indice.txt
```

5. Volver a comprobar una vez más el estado del repositorio:

```
git status
```

Ejercicio 4

1. Realizar un commit de los cambios:

```
git commit -m "Añadido índice del libro."
```

2. Ver el estado del repositorio:

```
git status
```

Ejercicio 5

1. Modificar el fichero indice.txt:

```
echo -e "Capítulo 1: Introducción a Git  
  
Capítulo 2: Flujo de trabajo básico Capítulo 3: Gestión de ramas Capítulo 4:  
Repositorios remotos" > indice.txt
```

2. **Mostrar los cambios respecto a la última versión:** `bash`

```
git diff
```

3. Hacer un commit de los cambios:

```
git add indice.txt  
git commit -m "Añadido capítulo 3 sobre gestión de ramas"
```

Ejercicio 6

1. Mostrar los cambios de la última versión respecto a la anterior:

```
git log --oneline  
git diff HEAD^ HEAD
```

2. Cambiar el mensaje del último commit:

```
Copiar  
git commit --amend -m "Añadido capítulo 3 sobre gestión de ramas al índice."
```

3. Volver a mostrar los últimos cambios del repositorio:

```
git log --oneline
```

2. Ejercicios de manejo del historial de cambios

Ejercicio 1

1. Mostrar el historial de cambios del repositorio:

```
git log
```

2. Crear la carpeta capítulos y el fichero capítulo1.txt:

```
mkdir capitulos  
echo "Git es un sistema de control de versiones ideado por Linus Torvalds." >  
capitulos/capitulo1.txt
```

3. Añadir los cambios a la zona de intercambio temporal:

```
git add capitulos/capitulo1.txt
```

4. Hacer un commit de los cambios:

```
git commit -m "Añadido capítulo 1."
```

5. Volver a mostrar el historial de cambios del repositorio:

```
git log
```

Ejercicio 2

1. Crear el fichero capítulo2.txt en la carpeta capitulos:

```
echo "El flujo de trabajo básico con Git consiste en: 1- Hacer cambios en el repositorio. 2-  
Añadir los cambios a la zona de intercambio temporal. 3- Hacer un commit de los cambios."  
> capitulos/capitulo2.txt
```

2. Añadir los cambios a la zona de intercambio temporal:

```
git add capitulos/capitulo2.txt
```

3. Hacer un commit de los cambios:

```
git commit -m "Añadido capítulo 2."
```

- Mostrar las diferencias entre la última versión y dos versiones anteriores:

```
git diff HEAD~2 HEAD
```

(Esto mostrará las diferencias entre la última versión y la versión anterior a la segunda más reciente.)

Ejercicio 3

- Crear el fichero capitulo3.txt en la carpeta capitulos:

```
echo "Git permite la creación de ramas lo que permite tener distintas versiones del mismo proyecto y trabajar de manera simultanea en ellas." > capitulos/capitulo3.txt
```

- Añadir los cambios a la zona de intercambio temporal:

```
git add capitulos/capitulo3.txt
```

- Hacer un commit de los cambios:

```
git commit -m "Añadido capítulo 3."
```

- Mostrar las diferencias entre la primera y la última versión del repositorio:

```
git diff HEAD~2 HEAD
```

(Por defecto, si has hecho un commit por cada capítulo, HEAD~2 será la versión más antigua y HEAD será la más reciente.)

Ejercicio 4

- Añadir al final del fichero indice.txt:

```
echo "Capítulo 5: Conceptos avanzados" >> indice.txt
```

- Añadir los cambios a la zona de intercambio temporal:

```
git add indice.txt
```

- Hacer un commit de los cambios:

```
git commit -m "Añadido capítulo 5 al índice."
```

- Mostrar quién ha hecho cambios sobre el fichero indice.txt:

```
git log -- indice.txt
```

3. Ejercicios de deshacer cambios

Ejercicio 1

1. Eliminar la última línea del fichero indice.txt:

```
sed -i '$d' indice.txt
```

2. Guardar los cambios:

```
git add indice.txt  
git commit -m "Eliminar la última línea de indice.txt"
```

3. Comprobar el estado del repositorio:

```
git status
```

4. Deshacer los cambios realizados en el fichero indice.txt:

```
git checkout HEAD -- indice.txt
```

5. Volver a comprobar el estado del repositorio:

```
git status
```

Ejercicio 2

1. Eliminar la última línea del fichero indice.txt:

```
sed -i '$d' indice.txt
```

2. Guardar los cambios:

```
git add indice.txt
```

3. Comprobar de nuevo el estado del repositorio:


```
git status
```

4. Quitar los cambios de la zona de intercambio temporal, pero mantenerlos en el directorio de trabajo:

```
git reset HEAD -- indice.txt
```

5. Comprobar de nuevo el estado del repositorio:

```
git status
```

6. Deshacer los cambios realizados en el fichero indice.txt para volver a la versión anterior:

```
git checkout HEAD -- indice.txt
```

7. Volver a comprobar el estado del repositorio:

```
git status
```

Ejercicio 3

1. Eliminar la última línea del fichero indice.txt:

```
sed -i '$d' indice.txt
```

2. Eliminar el fichero capitulos/capitulo3.txt:

```
rm capitulos/capitulo3.txt
```

3. Añadir un fichero nuevo capitulos/capitulo4.txt vacío:

```
touch capitulos/capitulo4.txt
```

4. Añadir los cambios a la zona de intercambio temporal:

```
git add indice.txt capitulos/capitulo3.txt capitulos/capitulo4.txt
```

5. Comprobar de nuevo el estado del repositorio:

```
git status
```

6. Quitar los cambios de la zona de intercambio temporal, pero mantenerlos en el directorio de trabajo:

```
git reset HEAD -- .
```

7. Comprobar de nuevo el estado del repositorio:

```
git status
```

8. Deshacer los cambios realizados para volver a la versión del repositorio:

```
git checkout HEAD -- .
```

9. Volver a comprobar el estado del repositorio:

```
git status
```

Ejercicio 4

1. Eliminar la última línea del fichero indice.txt:

```
sed -i '$d' indice.txt
```

2. Eliminar el fichero capitulos/capitulo3.txt:

```
rm capitulos/capitulo3.txt
```

3. Añadir los cambios a la zona de intercambio temporal y hacer un commit con el mensaje "Borrado accidental.":

```
git add indice.txt capitulos/capitulo3.txt  
git commit -m "Borrado accidental."
```

4. Comprobar el historial del repositorio:

```
git log
```

5. Deshacer el último commit pero mantener los cambios anteriores en el directorio de trabajo y la zona de intercambio temporal:

```
git reset --soft HEAD~1
```

6. Comprobar el historial y el estado del repositorio:

```
git log  
git status
```

7. Volver a hacer el commit con el mismo mensaje de antes:

```
git commit -m "Borrado accidental."
```

8. Deshacer el último commit y los cambios anteriores del directorio de trabajo volviendo a la versión anterior del repositorio:

```
git reset --hard HEAD~1
```

9. Comprobar de nuevo el historial y el estado del repositorio:

```
git log  
git status
```

4. Ejercicios de gestión de ramas

Ejercicio 1

1. Crear una nueva rama llamada bibliografia:

```
git checkout -b bibliografia
```

2. Mostrar las ramas del repositorio:

```
git branch
```

Ejercicio 2

1. Crear el archivo capitulos/capitulo4.txt y añadir el contenido:

```
mkdir -p capitulos # Crear el directorio si no existe  
echo "En este capítulo veremos cómo usar GitHub para alojar repositorios en remoto." >  
capitulos/capitulo4.txt
```

2. Añadir los cambios a la zona de intercambio temporal:

```
git add capitulos/capitulo4.txt
```

3. Hacer un commit con el mensaje "Añadido capítulo 4":

```
git commit -m "Añadido capítulo 4."
```

- Mostrar la historia del repositorio incluyendo todas las ramas:

```
git log --oneline --all --graph
```

Ejercicio 3

- Cambiar a la rama bibliografia:

```
git checkout bibliografia
```

- Crear el archivo bibliografia.txt y añadir la referencia:

```
echo "Chacon, S. and Straub, B. Pro Git. Apress." > bibliografia.txt
```

- Añadir los cambios a la zona de intercambio temporal:

```
git add bibliografia.txt
```

- Hacer un commit con el mensaje "Añadida primera referencia bibliográfica":

```
git commit -m "Añadida primera referencia bibliográfica."
```

- Mostrar la historia del repositorio incluyendo todas las ramas:

```
git log --oneline --all --graph
```

Ejercicio 4

- Fusionar la rama bibliografia con la rama master (sigue estos pasos, asegúrate de estar en la rama master):

```
git checkout master  
git merge bibliografia
```

- Mostrar la historia del repositorio incluyendo todas las ramas:

```
git log --oneline --all --graph
```

- Eliminar la rama bibliografia:

```
git branch -d bibliografia
```

- Mostrar de nuevo la historia del repositorio incluyendo todas las ramas:

```
git log --oneline --all --graph
```

Ejercicio 5

- Crear la rama bibliografia:

```
git checkout -b bibliografia
```

- Cambiar el fichero bibliografia.txt para que contenga las siguientes referencias:

```
echo -e "Scott Chacon and Ben Straub. Pro Git. Apress.\nRyan Hodson. Ry's Git Tutorial. Smashwords (2014)" > bibliografia.txt
```

- Añadir los cambios a la zona de intercambio temporal y hacer un commit con el mensaje "Añadida nueva referencia bibliográfica":

```
git add bibliografia.txt  
git commit -m "Añadida nueva referencia bibliográfica."
```

- Cambiar a la rama master:

```
git checkout master
```

- Cambiar el fichero bibliografia.txt para que contenga las siguientes referencias:

```
echo -e "Chacon, S. and Straub, B. Pro Git. Apress.\nLoeliger, J. and McCullough, M. Version control with Git. O'Reilly." > bibliografia.txt
```

- Añadir los cambios a la zona de intercambio temporal y hacer un commit con el mensaje "Añadida nueva referencia bibliográfica":

```
git add bibliografia.txt  
git commit -m "Añadida nueva referencia bibliográfica."
```

- Fusionar la rama bibliografia con la rama master:

```
git merge bibliografia
```

- Resolver el conflicto editando manualmente bibliografia.txt para que contenga las referencias correctas y luego añadir y hacer un commit:

```
# Abrir el archivo en un editor de texto, resolver el conflicto y guardar  
git add bibliografia.txt  
git commit -m "Resuelto conflicto de bibliografía."
```

9. Mostrar la historia del repositorio incluyendo todas las ramas:

```
git log --oneline --all --graph
```

5. Ejercicios de repositorios remotos

Ejercicio 1

1. **Crear un nuevo repositorio público en GitHub:**

- Inicia sesión en tu cuenta de GitHub.
- Haz clic en el botón "+" en la esquina superior derecha y selecciona "Nuevo repositorio".
- Nombralo libro-git, selecciona "Público" y haz clic en "Crear repositorio".

2. **Añadir el repositorio a tu repositorio local:**

- Abre la terminal o línea de comandos.
- Navega hasta el directorio de tu repositorio local donde tienes el libro. (Ejemplo: cd ruta/al/repositorio)
- Ejecuta el siguiente comando para añadir el repositorio remoto:

```
git remote add origin https://github.com/tu_usuario/libro-git.git
```

3. **Mostrar todos los repositorios remotos configurados:**

- Ejecuta este comando:

```
git remote -v
```

- Esto te mostrará los repositorios remotos configurados.

Ejercicio 2

1. Añadir los cambios del repositorio local al repositorio remoto:

- Asegúrate de tener cambios que desees añadir. Usa:

```
git add .
```

- Luego, haz un commit:

```
git commit -m "Descripción de los cambios"
```

- Finalmente, sube los cambios al repositorio remoto con:

```
git push origin main
```

- Asegúrate de reemplazar main con la rama correspondiente si no es main.

2. Acceder a GitHub y comprobar que se han subido los cambios:

- Dirígete a GitHub y abre el repositorio libro-git.
- Puedes comprobar el historial de versiones al hacer clic en "Commits" en la parte superior.

Ejercicio 3

1. Colaborar en el repositorio remoto libro-git de otro usuario:

- Clona el repositorio del usuario (asumiendo que tienes la URL):

```
git clone https://github.com/otro_usuario/libro-git.git
```

- Navega a este nuevo directorio:

```
cd libro-git
```

2. Añadir el fichero autores.txt:

- Crea el archivo autores.txt y añade tu nombre y correo electrónico:

```
echo "Tu Nombre <tucorreo@example.com>" > autores.txt
```

3. Añadir los cambios a la zona de intercambio temporal:

```
git add autores.txt
```

4. Hacer un commit:

```
git commit -m "Añadido autor."
```

5. Subir los cambios al repositorio remoto:

```
git push origin main
```

Ejercicio 4

1. Hacer una bifurcación del repositorio remoto asalber/libro-git:

- Ve al repositorio asalber/libro-git en GitHub y haz clic en "Fork" (Bifurcar).

2. Clonar el repositorio creado en tu cuenta de GitHub:

- Usa la URL de tu bifurcación:

```
git clone https://github.com/tu_usuario/libro-git.git
```

- Navega al directorio del repositorio clonado:

```
cd libro-git
```

3. Crear una nueva rama llamada autoria y activarla:

```
git checkout -b autoria
```

4. Añadir el nombre del usuario y su correo al fichero autores.txt:

- Abre autores.txt con un editor y añade tu nombre y correo:

```
echo "Tu Nombre <tucorreo@example.com>" >> autores.txt
```

5. Añadir los cambios a la zona de intercambio temporal:

```
git add autores.txt
```

6. Hacer un commit con el mensaje "Añadido nuevo autor.":

```
git commit -m "Añadido nuevo autor."
```

7. Subir los cambios de la rama autoria al repositorio remoto en GitHub:

```
git push origin autoria
```


8. Hacer un Pull Request de los cambios en la rama autoria:

- Ve a tu repositorio bifurcado en GitHub.
- Verás una opción para "Compare & pull request" después de subir la rama.
- Haz clic en "Create pull request" y completa los detalles.